Issue #64
Automated scenario execution

LiDRON Autonomous Systems | October 18, 2026

Change report and tester handoff

Implemented a single-command simulation lifecycle using the existing startup script, a standard-library Python host runner, and a ROS helper inside the autonomy container. The implementation sends the configured destination and starts the mission after readiness checks, then tears down the stack after completion, failure, timeout or cancellation.

What changed

- scripts/run_scenario.py: scenario validation, exclusive stack ownership, bounded processes, cleanup, exit codes and atomic result output.

- scripts/scenario_ros.py: node/service/input readiness, destination submission, mission enable and terminal-state observation.

- scenarios/clear_path.json: default world/spawn and the design example destination (8 m north, 0 m east, down -5 m).

- scripts/simulation.sh: added a running-container query for ownership checks.

- tests/: 14 local tests using fake processes and ROS doubles.

- docs/scenario-testing.md and README: run instructions, expected outcomes, negative tests and evidence to return. Generated results are ignored by Git.

Verification and limits

All 14 new local tests passed. No real Gazebo/PX4 mission has been run here. Flight, route clearance, fresh spawn, teardown and repeated execution require the lab checks in the following guide. Existing pytest-based ROS/perception tests were not run because pytest and the ROS environment are unavailable in this session.

Execution scope

COMPLETED means the runner observed the terminal COMPLETE state after accepted commands. It is not a validation PASS. Detailed path, arrival, state-order and obstacle evaluation remain for the validation work. Custom worlds, spawn settings and obstacle injection are rejected in this initial version.

References: Issue #64 and Issue #62 design attachment.

Branch: `codex/issue-64-scenario-runner`. Scope: execution only.

Changes and intended behavior

`python3 scripts/run_scenario.py scenarios/clear_path.json` validates the selected JSON scenario, refuses an already running stack, starts `scripts/simulation.sh up`, waits for ROS readiness, sends a local NED destination, enables the mission, observes terminal states, and runs `simulation.sh down` automatically. The runner uses a per-user lock to prevent two runners from sharing the fixed container names. Run one scenario at a time from a dedicated Simulation checkout.

The ROS helper subscribes to `/autonomy/state` before sending any mission command. Readiness requires the `perception`, `mission`, and `diagnostics` nodes, a recent `IDLE` state, all three required services, and a successful `/autonomy/preflight_check`. That existing service checks fresh camera, LiDAR, GPS and PX4 messages and sensor frame IDs. Destination must return `accepted=true`; enable must return `success=true`.

A fresh default world/spawn is obtained by full stack teardown, followed by a new launch for the next run. The runner does not edit the Simulation repository or perform an in-place Gazebo world reset. It also does not remove Docker volumes. Verify the initial spawn visually during lab testing.

Configuration and execution-status contract

The example follows the configuration concepts in the design attached to issue #62: name/version, environment, UAV start mode, mission, timeouts and expectations. Version 1 supports `environment={"profile":"default","obstacles":[]}`, `uav={"start_mode":"default"}`, and `mission.frame="local_ned"` only. Unsupported worlds, spawn modes, obstacles, frames and malformed/nonfinite numbers are rejected before launch. Default world selection and spawn remain owned by Simulation. The built-in validator needs only Python's standard library; no host ROS dependency or external JSON Schema package is required. Required fields and supported values are checked directly rather than introducing a second schema definition.

Change `mission.destination.north/east/down` in meters. Negative down means above the NED origin. The example is north=8, east=0, down=-5. Environment and UAV configuration explicitly select the existing defaults; custom world/spawn and obstacle injection are future extensions.

`timeouts.startup_s` covers launch/build and readiness; `mission_s` starts immediately before the enable request; `cleanup_s` bounds the teardown command. All use monotonic wall time, independent of paused simulation time. Allow extra startup time for a first uncached Docker/colcon build (e.g. 900 seconds). The helper receives the remaining startup budget when invoked; container startup adds small scheduling overhead. An outer watchdog also bounds a stuck helper. Process termination can add up to two seconds before a forced kill.

execution_status	Meaning	Exit code
COMPLETED	Observed COMPLETE after accepted mission commands	0
FAILED	Observed ABORT or HOLD during the mission	1
TIMEOUT	Mission deadline expired	124
ERROR	Invalid configuration, startup, service, or cleanup failure	2
CANCELLED	Host runner received Ctrl-C or SIGTERM	130

Startup timeout is `ERROR`, `reason=TIMEOUT`, `phase=startup`. A mission timeout is `TIMEOUT`, `reason=TIMEOUT`, `phase=mission`. Command rejection/infrastructure errors use `INFRASTRUCTURE_ERROR`; ABORT/HOLD use `MISSION_FAILURE`. Cleanup failure changes the final status to `ERROR/CLEANUP_ERROR` and retains `original_status` and `original_reason`. Cleanup is attempted even after partial startup or cancellation. An already running stack is refused and is not stopped.

`COMPLETED` is not a validation PASS. The `expected` section is checked for valid structure and contradictory required/forbidden states, then preserved for a future validator. This implementation does not evaluate path generation, arrival tolerance, state ordering, or obstacle behavior. Those belong to the validation issue.

Each launched run saves evidence under `results/<UTC timestamp>-<unique ID>/`:

- `scenario.json`: selected configuration snapshot.

- `result.json`: atomic final execution outcome after cleanup, duration, scenario,

run ID and configuration SHA-256. This is the validation component's entry point.

- `events.jsonl`: observed state changes and preflight/destination/enable responses,

with monotonic elapsed timestamps, once the ROS helper starts.

- `execution.json`: helper's provisional outcome before host cleanup.

- `stack.log`, `mission.log`, `cleanup.log`: launch, helper and teardown logs.

Results are ignored by Git. Invalid configurations print structured `CONFIG_INVALID` JSON and exit 2 before creating a run directory. Lock refusal also prints JSON. Neither condition launches or tears down containers.

Test 1: local checks without Gazebo

From the repository root:

```
python3 -m unittest discover -s tests -v
```

Expected: 14 tests pass. These use fake processes and ROS doubles; they do not start containers or prove that a simulated mission can fly. They cover readiness and command ordering, rejected commands, failure signals, mission/startup timeouts, cancellation, partial launch failure, cleanup failure, active stack refusal and configuration rejection.

Test 2: lab startup, automatic flight and teardown

Requirements: Ubuntu 24.04 (native, WSL2, or UTM as described in README), Python 3, Docker/Compose, working Gazebo display, and the matching Simulation checkout. The branch must be shared with the tester before they can fetch it.

```
git fetch origin
git switch codex/issue-64-scenario-runner
export SIMULATION_ROOT=/absolute/path/to/Simulation
./scripts/simulation.sh check
python3 scripts/run_scenario.py scenarios/clear_path.json
runner_code=$?
echo "Runner exit: $runner_code"
./scripts/simulation.sh running
```

Expected before mission: all required containers launch, autonomy builds, and no manual destination/enable command is needed. In `events.jsonl`, `ready` precedes `destination` with accepted=true, which precedes `enable` with success=true.

Expected flight: default spawn, takeoff, motion toward 8 m north at 5 m altitude, landing, then COMPLETE. Inspect the route and landing surface in the selected world; this scenario's clear path is not yet confirmed in the lab.

Expected final outcome for a successful execution: exit 0, `execution_status=COMPLETED`, `last_state=COMPLETE`, `cleanup=ok`. `simulation.sh running` prints no container IDs. Inspect all logs if the outcome is different; do not treat an unverified COMPLETED as validation of route accuracy. On a slow initial build, copy the scenario to `/tmp/clear_path.json`, increase `timeouts.startup_s` to 900, and pass that file to the runner.

Test 3: mission timeout and fresh repeat run

Create a timeout scenario without editing the checked-in example:

```
python3 - <<'PY'
import json
from pathlib import Path
p = json.loads(Path('scenarios/clear_path.json').read_text())
p['name'] = 'mission_timeout'
p['timeouts']['startup_s'] = 900
p['timeouts']['mission_s'] = 0.1
Path('/tmp/mission_timeout.json').write_text(json.dumps(p))
PY
python3 scripts/run_scenario.py /tmp/mission_timeout.json
runner_code=$?
echo "Runner exit: $runner_code"
./scripts/simulation.sh running
```

Expected: preflight readiness first, then TIMEOUT during mission, exit 124, and cleanup=ok. No container IDs remain. The vehicle may have only just begun takeoff when the timeout tears down this simulation.

Run the normal scenario again. Expected: fresh default world/spawn and IDLE state, no old mission resumption, and a new result directory. Record both run IDs.

Test 4: startup timeout, cancellation and refusal

For startup timeout, create another temporary copy with startup_s=0.1 and mission_s=300. Run it as above. Expected: ERROR/TIMEOUT, phase=startup, exit 2, cleanup=ok after any attempted launch, and no running stack containers.

For cancellation, run the normal scenario and press Ctrl-C while startup or flight is active. Expected: CANCELLED, exit 130, cleanup=ok, and no running stack containers. Ctrl-C again during teardown may interrupt cleanup and produce ERROR/CLEANUP_ERROR; use one interrupt and wait for teardown.

For refusal, manually start `./scripts/simulation.sh up` in one terminal. Run the scenario runner in a second terminal. Expected: ERROR with a message that the stack is already running; the manually launched containers remain untouched. Stop that manual stack with `./scripts/simulation.sh down` before further tests.

Test 5: failure signal and invalid configuration

During a normal scenario run, after mission enable, use a second terminal to inject a terminal state into this simulation only:

```
docker exec lidron_autonomy bash -c '
  source /autonomy/install/setup.bash &&
  ros2 topic pub --once /autonomy/state std_msgs/msg/String "{data: HOLD}"
'
```

Expected: if the helper receives HOLD before another terminal state, it records FAILED/MISSION_FAILURE, last_state=HOLD, exits 1 and tears down the stack. This checks the execution failure interface; it does not validate a real vehicle fault. Publishing may fail when teardown starts; retain events.jsonl to show what arrived.

Create a temporary scenario with mission.frame set to `enu` (or a negative timeout). Expected: printed ERROR/CONFIG_INVALID, exit 2, no new run directory and no container changes. Do not modify production ROS topics or run these tests on physical hardware.

Evidence to send back

For each test, send the scenario file, console output/exit code, entire result folder, Simulation and Autonomous-Systems commit IDs (`git rev-parse HEAD` in each), OS/Docker versions, and a screenshot or short recording of spawn/flight/landing. Record expected vs actual, whether cleanup left containers running, and whether a second run started fresh. An infrastructure failure is useful evidence too.

Local checks passed on October 18, 2026. Real Docker/Gazebo/PX4 flight, route clearance, fresh spawn, and repeated lab execution remain pending. No live simulation success is claimed. Automatic teardown does not replace a controlled landing on hardware; this entry point is for the simulated stack only.